

Exact Machine Unlearning on Weightless Neural Networks

Anonymous Author(s)¹

¹Affiliation withheld for double-anonymous review

anonymous@example.org

Abstract. Privacy regulations grant individuals a right to erasure, so a model trained on personal data must be able to forget it, deleting a record cheaply, completely, and verifiably. These goals are hard to meet together: retraining from scratch is exact but expensive, cheaper methods leave residue that membership-inference attacks can recover, and accuracy benchmarks never score a model’s fitness for deletion. We show that weightless neural networks of the WiSARD family delete by construction, because training increments independent counters and deleting an example subtracts exactly its contribution, leaving the model identical to one that never saw it. We measure this on a synthetic, calibrated Where’s-Waldo benchmark with an open evaluation harness that scores retained accuracy, deletion cost, memory, and residual leakage against four unlearning baselines. Across class erasure, poison removal, deletion streams, and backdoor scrubbing, the weightless network deletes a class in 40 ms at 9.3 MB with membership inference at chance and 0.89 accuracy, while retraining spends 1.3 s and a data-removal-enabled forest matches 0.95 accuracy only at 13.9 s and 1.6 GB. Scoring deletion as a first-class property reveals weightless networks as the latency-first exact option for machine unlearning.

1. Introduction

Machine learning models trained on personal data inherit an obligation to forget it. Privacy regulations such as the Brazilian LGPD and the European GDPR grant individuals a right to erasure, and once a record has been absorbed into a trained model, removing it from storage is insufficient: the model must be restored to the state it would have reached without that record. This operation, machine unlearning [Cao and Yang 2015, Nguyen et al. 2022, Xu et al. 2023], also supports security, by excising a poisoned or backdoored example, and maintenance, by retiring stale or mislabeled data. A usable unlearning method must delete a record cheaply, completely, and verifiably.

These three requirements are hard to satisfy together. Retraining from scratch on the surviving data is complete and verifiable but pays the full training cost at every request; cheaper exact methods such as sharded retraining [Bourtole et al. 2021] and data-removal-enabled forests [Brophy and Lowd 2021] avoid that cost but still rebuild substructure or cache large statistics to stay exact, while approximate methods such as gradient ascent give up completeness, leaving residual traces that membership-inference attacks [Shokri et al. 2017, Chen et al. 2021] can recover. Accuracy benchmarks compound the problem, scoring only what a model predicts and never what it can forget,

so a model’s fitness for deletion never appears in the comparisons practitioners read. What the field lacks is the recognition that one model family already deletes by construction, together with a benchmark that treats deletion as a first-class property.

In principle, exactness is free for any model whose training decomposes additively over examples: if the trained state is the sum of independent per-example contributions, deleting an example is undoing its contribution, with no substructure to rebuild and no parameter to perturb. Weightless neural networks of the WiSARD family [Aleksander et al. 1984], descended from the n-tuple recognizer [Bledsoe and Browning 1959], are exactly such a model: each class is a set of memory nodes whose integer counters are incremented once per training example, independently and additively, so deleting an example decrements its counters and leaves the model bit-identical to one that never saw it. The `wisardpkg` library [Lima Filho et al. 2020] already exposes this operation, and additive models have long been known to admit exact unlearning in principle [Cao and Yang 2015]; what is missing is the recognition that this places weightless networks at an extreme of the unlearning trade space, and a benchmark that measures where. Measuring it needs a task with enough repeated, balanced identities to delete in volume and audit what remains. Where’s Waldo, a visual-search puzzle with a recurring cast of look-alikes, is a convenient such task, and we build a synthetic, calibrated version of it because the few annotated real pages do not provide that volume [Barthelme and Vidal 2021].

We locate weightless networks on the unlearning trade space using a synthetic benchmark and a shared evaluation harness. The generator emits per-identity-balanced instances with free localization labels, calibrated against real Where’s-Waldo statistics; the harness runs WiSARD against four baselines (gold-standard retraining, sharded retraining, data-removal-enabled forests, and gradient ascent) on common data and splits, scoring retained accuracy, deletion cost, memory, and residual leakage by membership-inference AUC. Across class erasure, poison removal, deletion streams, and backdoor removal, WiSARD deletes a class in 40 ms against the retrained baseline’s 1.3 s, at 9.3 MB of memory, with membership-inference AUC at chance and accuracy of 0.89 against the baseline’s 0.95. The data-removal-enabled forest matches that 0.95 under exact deletion but spends 13.9 s and 1.6 GB to reach it, occupying the accuracy-first extreme of the trade space while WiSARD occupies the latency-first extreme (Figure 2).

This paper makes three contributions:

- (i) a synthetic Where’s-Waldo benchmark generator that produces per-identity-balanced instances with localization labels, calibrated against real Where’s-Waldo statistics and released openly;
- (ii) the identification and first benchmark of weightless networks as an exact, residual-free unlearning substrate, evaluated against retraining, sharded retraining, data-removal-enabled forests, and gradient ascent, and audited by membership inference;
- (iii) an open evaluation harness implementing the four unlearning baselines and the membership-inference audit.

The paper is divided as follows. Section 2 reviews weightless networks, the

Where’s-Waldo task, and the machine-unlearning literature. Section 3 motivates and describes the synthetic benchmark. Section 4 establishes the weightless network as a competitive model on the task, fixing its input encoding and benchmarking a weightless localizer against a tuned random forest. Section 5 places the weightless network against the unlearning baselines. Section 6 examines threats to validity, and Section 7 concludes.

2. Related Work

This section surveys three bodies of work the paper draws on and one it must build past: weightless neural networks, the machine-unlearning literature, the membership-inference tools used to verify forgetting, and the Where’s-Waldo task as a benchmark.

Weightless neural networks recognize patterns with random-access memory rather than tunable weights: the n-tuple method of Bledsoe and Browning samples bit-tuples from the input and looks them up in tables [Bledsoe and Browning 1959], and the WiSARD architecture organizes those tables into one discriminator per class [Aleksander et al. 1984]. The `wisardpkg` library provides a maintained implementation [Lima Filho et al. 2020], and the regression WiSARD extends the model from classification to real-valued outputs [Lusquino Filho et al. 2020], which we use for localization. This literature consistently presents weightless models as fast, single-pass, online-trainable classifiers, and studies their accuracy and memory footprint. It does not study the consequence we exploit: because each table cell is an independent additive counter, the trained model is a sum of per-example contributions, and deletion is exact by construction rather than approximate.

Machine unlearning seeks the model that retraining on the surviving data would have produced, without paying the full retraining cost [Cao and Yang 2015, Nguyen et al. 2022, Xu et al. 2023]. Exact methods bound that cost structurally: SISA shards the data and retrains only the affected shard [Bourtoule et al. 2021], while data-removal-enabled (DaRE) forests cache split statistics so most deletions touch few subtrees [Brophy and Lowd 2021]; approximate methods instead perturb a trained model toward the unlearned state, for example by gradient ascent on the deleted example, trading exactness for speed. Each technique is engineered around a model class—shards for deep networks, cached statistics for forests, gradient steps for differentiable models—and the exact variants still reconstruct substructure or spend large memory to do so. A newer direction instead makes forgetting native to the architecture: MUNKEY deletes by removing instance keys from a memory-augmented transformer rather than by updating weights [Laguna et al. 2026]. Weightless networks share that goal of architecture-native deletion but reach it by construction, with a deletion that is an exact counter subtraction, bit-identical to never-training; we benchmark it against SISA, DaRE, gradient ascent, and gold-standard retraining on shared data and metrics.

Whether a deletion truly removed a record is itself measurable: membership-inference attacks ask whether a given example was in the training set, and a clean deletion should drive their advantage to chance [Shokri et al. 2017]. The strongest form calibrates each example against shadow models [Carlini et al. 2022], and for

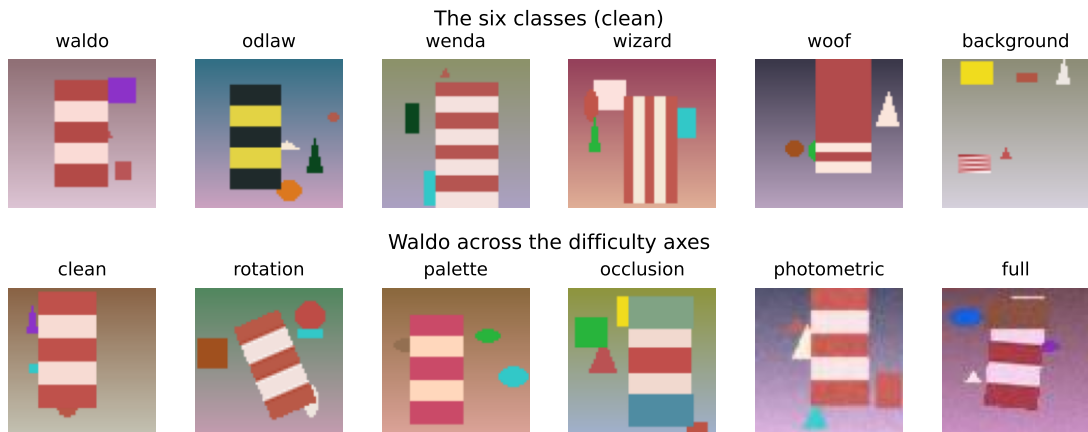


Figure 1. The synthetic benchmark. Top: one clean patch of each of the six classes—the target Waldo and four look-alikes that each differ from him on a single feature (Odlaw’s palette, Wenda’s stripe count, Wizard’s orientation, Woof’s partial band), plus a background class of distractors and striped clutter. Bottom: the same Waldo under each difficulty axis (rotation, palette jitter, occlusion, photometric degradation) and all of them combined.

unlearning in particular per-example audits are needed to avoid a false sense of privacy [Hayes et al. 2024], since unlearning can paradoxically increase leakage when the before- and after-deletion models are both exposed [Chen et al. 2021]. These attacks are usually mounted against deep networks and approximate unlearning, where residual traces are expected; we adopt the membership-inference AUC as the forgetting check for weightless deletion, where the additive structure predicts AUC at chance and the experiment confirms it.

Visual search in the style of Where’s Waldo? has been approached as a single-image detection problem, with machine-learning treatments that locate the character on a scanned page [Barthelmes and Vidal 2021]. These treatments optimize detection on the handful of published illustrations, where one labeled instance per page suffices. Our use of the task is different: we need many balanced instances of each character and a known target location in every scene, so that identities can be deleted in volume and audited and a weightless localizer can be scored on a level field, which the few annotated real pages cannot provide. In contrast to all of the above, this paper builds a synthetic, calibrated Where’s-Waldo benchmark and treats weightless networks as an exact, residual-free unlearning substrate, positioning them as a latency-first peer to accuracy-first exact methods such as DaRE rather than as a replacement for them.

3. A Synthetic Where’s-Waldo Benchmark

Where’s Waldo is a recognizable visual-search puzzle in which the reader must find one unremarkable figure among hundreds, aided and hindered by a recurring cast of look-alikes (Odlaw, Wenda, Wizard Whitebeard, and the dog Woof) that the artist draws to resemble Waldo. That built-in cast of near-duplicates makes the puzzle a convenient testbed for a recognizer, and machine-learning treatments of it exist [Barthelmes and Vidal 2021]. We use a synthetic version rather than the

books because the experiments need controlled material the published pages cannot give: many balanced instances of each character, difficulty we can vary, and a known target location in every scene. The largest annotated real collection, Hey-Waldo [Constantinou 2017], marks the target in only 14 scenes at one instance per page, which is enough to calibrate against but not to run an experiment on, so we measure those pages and match the generator to them: Waldo’s red is a muted (185, 84, 78) and his white a cream (247, 225, 217), his width a median 1.4% of the page at aspect 0.52, with red covering roughly 17% of the whole page and white 22%. Because the page is as red-and-white as Waldo himself, colour alone cannot locate him, so the benchmark inherits a task that rewards stripe structure over palette.

The generator abstracts each character to the one structural trait that survives a small patch. In the books the characters are told apart by fine facial detail, which is lost once a figure shrinks to the few-dozen pixels a recognizer sees, so we keep instead the trait that does survive: a character’s palette, stripe orientation, stripe layout, or completeness. This gives six classes (Figure 1): the target Waldo and four look-alikes (Odlaw in a black-and-yellow palette, Wenda with a different stripe count, Wizard with vertical stripes, and Woof showing only a partial striped band), plus a background class. Each look-alike differs from Waldo on exactly one feature, which makes it a controlled hard negative, and the background class is not blank canvas but distractors and striped clutter (barber poles, awnings, flags) that share Waldo’s colour and stripe texture while differing in scale and shape, so a model cannot succeed by flagging any red-and-white stripe.

From this shared machinery the generator emits two products. For recognition it draws balanced 48×48 patches, one character or none per patch, which the encoding study and the unlearning experiments consume. For localization it composes cluttered scenes with one Waldo at a known position among friends and clutter, returning that position as the label against which the sliding-window localizer of Section 4 is scored. Both products share four difficulty axes (rotation, palette jitter, occlusion, and a photometric degradation that mimics a photographed page) applied one at a time or together, so every comparison runs from clean to severe and difficulty is set to be hard for all models rather than tuned against one.

A reduced task would be worthless if a simple rule could read the class off each patch, so we check. A depth-three decision tree over aggregate colour and stripe statistics reaches only about 0.5 macro-F1 on clean patches, while the same patches let a model trained on raw pixels reach about 0.95, and the gap holds as difficulty rises, because the distinguishing traits (how many stripes, where the band sits, how the pattern is arranged) live in a spatial layout that aggregate statistics discard. We release the generator openly, so the benchmark is reproducible from a seed. Because the same identities recur in controlled volume, this generator supplies both the recognition workloads that the unlearning study deletes from in Section 5 and the scenes that the localizer reads in Section 4.

4. Encoding and Localization

Section 4.1 fixes the binary encoding a weightless network reads and compares the tuned network against strong classical baselines on recognition; Section 4.2 builds

a weightless localizer and benchmarks it against a tuned random forest. Together they establish the weightless network as a competitive model on the task before the unlearning study.

4.1. Encoding and recognition

A weightless network has no weights to compensate for a poor representation: each RAM memorizes the bit-tuples it saw for a class, and recognition is a vote over RAM lookups, so encoding the input is the modelling. We binarize each colour channel with a thermometer code, group the resulting position-tagged bits into RAM address tuples, and because the thermometer type and resolution and the tuple size all have interior optima, choose them by a grid search over three seeds rather than by default; the search settles on a Gaussian-spaced thermometer of 13 bits per channel with address size 12, which we fix for the comparison.

With the encoding fixed, we place the tuned weightless network beside logistic regression and a random forest, each tuned over its own hyperparameters on the raw pixels so that no baseline is a strawman. On clean patches the random forest leads at 0.95 macro-F1, the weightless network follows at 0.90, and logistic regression trails at 0.84; under full difficulty the three fall together to 0.77, 0.72, and 0.67. The weightless network is competitive on accuracy without dominating any axis: it is heavier (about 10 MB against the forest’s 4.4 MB and logistic regression’s 81 KB) and slower (about 1 ms per patch against the forest’s 0.07 ms), so the benchmark does not secretly favour it.

4.2. Localization

Localization needs a different formulation than regressing coordinates. A global Regression-WiSARD that reads the whole scene and predicts the target’s centre fails, collapsing to the mean training position at 31 px error, no better than predicting that mean, because the random bit-sampling destroys spatial locality and a small target barely moves the scene’s global statistics; position is a where, not a what. We instead slide a window across the scene, score each window for how well it is centred on the target with a Regression-WiSARD, and take the peak of the resulting score map as the predicted location.

To compare models rather than features, the weightless localizer and a random-forest window classifier read the same compact, hand-designed encoder; a random forest on raw pixels reaches 5.3 px, so the encoder helps it too and is reported as shared preprocessing. On a level field the weightless localizer ties the tuned forest: 3.8 px median error with 97% of targets within 12 px, against the forest’s 3.7 px and 99%, at 540 KB against the forest’s 1.6 MB, while a HOG+SVM detector trails at 19 px. Neither learner dominates the accuracy-versus-memory trade-off.

The tie is stable under difficulty. Retraining both models at each difficulty level, the weightless localizer is slightly ahead on clean scenes (3.1 px against 4.1 px) and the forest slightly ahead at the hardest setting (22.1 px against 23.6 px), with neither collapsing in between. Across recognition and localization the weightless network is therefore a competitive model on this task, which is the premise the

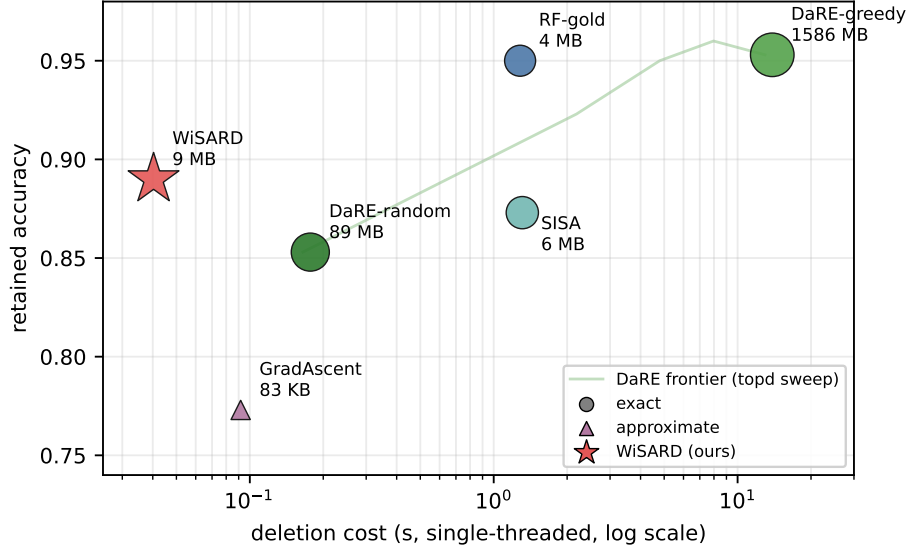


Figure 2. Deletion cost versus retained accuracy for a right-to-be-forgotten class erasure, timed single-threaded. Marker area encodes the memory a method holds to support deletion; circles are exact methods and the triangle is the approximate baseline. The weightless network occupies the latency-first corner (40 ms, 0.89, 9.3 MB) and the greedy DaRE forest the accuracy-first corner (13.9 s, 0.95, 1.6 GB); sweeping the DaRE frontier, no setting reaches both the weightless network’s cost and its accuracy.

next section depends on: the deletion advantage we measure is the advantage of a credible model, not of a weak one.

5. Weightless Networks as an Exact Unlearning Substrate

Section 5.1 explains why deletion is exact for a weightless network and how that exactness can be certified; Section 5.2 states the metrics and baselines; Section 5.3 reports four deletion scenarios. Figure 2 previews the result as a deletion trade surface on which the weightless network and the exact baselines occupy different corners.

5.1. Exact deletion by construction

A weightless network trains by addition. Each training example increments a fixed set of RAM counters, one per address it activates, independently of every other example, so the trained model is the sum of its per-example contributions. Deleting an example is therefore subtraction: decrementing exactly the counters it incremented returns the model to the state it would have reached without that example, bit for bit. The operation is untrain, already provided by the `wisardpkg` library; it is exact by construction, with no substructure to rebuild as in a forest and no weight to nudge as in gradient descent.

This exactness can be certified directly. Because deletion restores the counters to their never-incremented values, the vote memory after unlearning a batch is identical to that of a model that never saw the batch, and the two can be compared cell by cell; in our experiments the maximum counter difference is zero. No weighted

model offers this check: the influence of one example cannot be read off a forest’s splits or a network’s weights, so its removal can only be trusted by construction, retrained-and-compared at full cost, or probed statistically. We adopt the statistical probe, the membership-inference attack, as the common currency that also scores the approximate baseline.

5.2. Metrics and baselines

We ask three questions of each deletion: whether the model behaves as if the data were never seen, what the deletion costs, and whether retained accuracy is preserved. Forgetting is read two ways: the forget effect, the model’s accuracy on the deleted data, which a correct deletion drives to chance; and the membership-inference AUC of a confidence-threshold attack, which separates the deleted examples from fresh never-seen examples of the same kind using the unlearned model’s confidence, where 0.5 means indistinguishable and above 0.5 a residual leak. Deletion cost is the wall-clock of the deletion alone, measured single-threaded so it reflects algorithmic work rather than core count, together with the memory a method holds to support deletion; retained utility is test accuracy on the data kept.

We compare the weightless network against four baselines on a shared six-class corpus: retraining from scratch on the surviving data, the exact gold standard; SISA, which shards the data and retrains only affected shards; data-removal-enabled (DaRE) forests in their greedy and random regimes, the purpose-built exact-forest method, matched to the gold-standard forest and applied one-vs-rest; and gradient ascent, an approximate method that raises the deleted example’s loss in place. The first three are exact; gradient ascent is not.

5.3. Four deletion scenarios

Erasing a class is the first request: all Odlaw training rows are deleted, and the model must keep the other five classes intact. Every exact method removes Odlaw completely and drives its membership AUC to 0.5, so the separation is cost (Table 1). The weightless network empties Odlaw’s discriminator in 40 ms at its 9.3 MB footprint; retraining rebuilds all 300 trees in 1.28 s, SISA retrains every shard in 1.31 s (class deletion is its worst case, the class being spread across all shards), and the DaRE forest needs 13.9 s and 1.6 GB of cached split statistics to keep the deletion both exact and as accurate as the gold standard. Retained accuracy is 0.89 for the weightless network against the gold standard’s 0.95, while gradient ascent is cheap at 92 ms but approximate, paying in utility here as it falls to 0.77. The certificate holds throughout: the scrubbed vote memory is identical to a never-trained one, and Figure 3 shows the deletion from inside both models.

Removing a poisoned contribution is more discriminating, because the class survives and membership becomes a real question. We inject 100 Wizard patches mislabelled as Waldo, then delete exactly those rows; before deletion the attack separates them well above chance, confirming the model memorised them. The weightless network erases the 100 counters in milliseconds and collapses the membership AUC to 0.5, restoring clean accuracy, while gradient ascent, at comparable cost, leaves the AUC above 0.5—the residual leak the exactness guarantee exists to prevent. The exact methods trace a trade surface, with the weightless network at

Table 1. Class erasure (delete all Odlaw rows), single-threaded. Cost is the deletion wall-clock; accuracy is on the five kept classes; memory is what each method holds to support deletion; a membership AUC of 0.5 is clean forgetting. Best per column in bold.

Method	Cost (s)	Acc.	Mem. (MB)	Exact	MIA
WiSARD (ours)	0.040	0.89	9.3	yes	0.50
Retrain (gold)	1.28	0.95	3.8	yes	0.50
SISA	1.31	0.87	6.3	yes	0.50
DaRE-greedy	13.87	0.95	1585.7	yes	0.50
DaRE-random	0.18	0.85	89.1	yes	0.50
Gradient ascent	0.09	0.77	0.08	no	0.48

the latency-first end (tens of milliseconds, 0.89 accuracy, 9.3 MB) and the greedy DaRE forest at the accuracy-first end (matching the gold standard’s 0.95, but at roughly 14 s and 1.6 GB); sweeping DaRE’s full accuracy-cost frontier, no setting is at once as accurate as the gold standard and as cheap as the weightless deletion.

The cost that matters in deployment is not one deletion but a stream of them. Replaying 30 deletion requests, each removing one contributor’s rows, the weightless network’s cumulative cost stays flat, because each untrain costs in proportion to the rows deleted and is independent of corpus size and history; the thirtieth request costs what the first did, 0.25 s in total. The exact baselines accumulate: best-case SISA, sharded by contributor so each request hits one shard, still spends 2.5 s, the DaRE forest in its random regime 1.4 s, and gold-standard retraining 27 s, because each request rebuilds a 300-tree model. The constant-versus-linear gap is the deployment argument, and it only widens over a model’s lifetime of accumulating requests.

The same primitive defends a detector, not only a classifier. We poison a sliding-window Waldo detector with a distinctive trigger mislabelled as the target, so that before scrubbing every trigger patch is detected as Waldo and the trigger fires the detector in every scene. A single untrain of the poisoned windows removes the backdoor exactly, dropping trigger detection from 1.00 to 0.00 in milliseconds while genuine Waldo detection is untouched. Removing a trojan is the same counter-subtraction as honouring a deletion request, and it extends the exact guarantee from patch classification to spatial detection.

6. Discussion

The weightless network’s advantage is a position on the deletion axis, not a new accuracy frontier. On static accuracy the random forest leads and the weightless network follows, and the same additive structure explains both: independent per-example counter updates make deletion a constant-time primitive, and they also cap capacity below a forest that splits on feature interactions. What the method offers is exact, millisecond, flat-memory deletion at competitive accuracy, a place the other exact methods reach only by spending seconds or gigabytes; the greedy DaRE forest reaches the same exactness at the accuracy-first end, so the two are complementary deployment priorities rather than competitors.

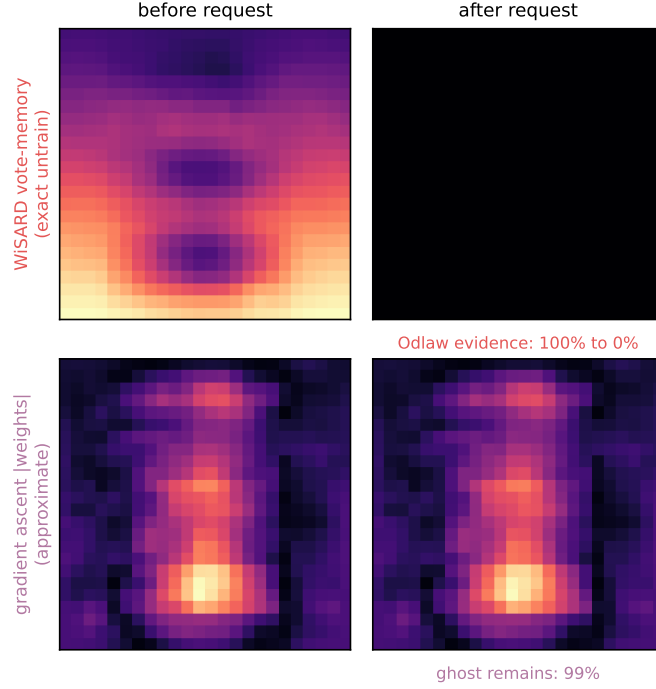


Figure 3. Forgetting Odlaw, seen from inside each model. Top: the weightless network’s back-projected vote memory for Odlaw is a structured imprint before the request and identically zero after untrain (1.16M votes to 0), neighbouring classes unchanged. Bottom: gradient ascent can only push weights down, leaving 99% of the Odlaw evidence as a structured ghost—the residue that surfaces as a membership-inference leak once the class survives deletion (the poison scenario).

Several limitations bound these claims. The benchmark is synthetic, calibrated to real Where’s-Waldo pages but abstracting each character to a structural trait, so transfer to photographed pages is untested, and the study covers one task family. The weightless network’s bit-to-tuple mapping is unseeded, so its recognition accuracy fluctuates between about 0.88 and 0.92 across runs, and we report single-run values rather than multi-seed intervals. Its memory is competitive but not minimal, heavier than a plain forest, and its memory standing is an advantage only against the deletion-enabled DaRE forest, not against a forest that never has to forget. Membership inference is a single probe of forgetting, and ours is a confidence-threshold attack: for the weightless network the bit-identity certificate is the stronger guarantee, but it exists only because training is additive, and for the approximate baseline a stronger per-example attack, such as the likelihood-ratio attack [Carlini et al. 2022] or its unlearning-specific variant [Hayes et al. 2024], could surface residue our AUC does not.

The main threat to external validity is the gap between synthetic and real pages, but it bears on the benchmark, not the mechanism. Exactness follows from additive training, not from anything specific to Waldo: wherever a weightless recognizer is an appropriate model, deletion is the same counter subtraction, with the same certificate and the same cost. The result to carry forward is therefore narrow but portable, a weightless network that deletes exactly and cheaply by construction,

established here on a controlled task before it is taken to messier ones.

7. Conclusion

This paper presented weightless neural networks as an exact machine-unlearning substrate, together with a synthetic Where’s-Waldo benchmark and an open evaluation harness that scores deletion as a first-class property. The experiments demonstrate that, because training is additive, deletion is a counter subtraction that is exact by construction and certifiable by direct comparison: across class erasure, poison removal, deletion streams, and backdoor scrubbing, the weightless network deleted in milliseconds at a flat memory budget with membership inference at chance, while the exact baselines reached the same exactness only at seconds-to-gigabyte cost and the approximate baseline left a measurable leak. One can therefore see that accuracy benchmarks, which score only what a model predicts and never what it can forget, keep a model family that forgets by construction invisible in the comparisons practitioners read; placing deletion on the same axis as accuracy makes that family legible and turns the right to be forgotten into a model-selection criterion, on which a weightless network is the latency-first exact option beside the accuracy-first forest.

Three directions follow. First, the accuracy gap to the random forest is the price of a low-capacity additive model, and closing it without surrendering exact deletion is an open problem: higher-capacity additive models, for instance ensembles of weightless discriminators that keep each update independent, could move the latency-first corner toward the forest’s accuracy while leaving deletion a counter subtraction. Second, the same additive primitive that deletes on request also unlearns online: applied to each mistake as appearance drifts, it lets the model track change at microsecond cost, a continual-unlearning use we have only previewed here and that deserves a full prequential study. Third, because the mechanism is independent of the benchmark, the natural next test is transfer—training on the calibrated synthetic pages and deleting on real photographed ones—and, beyond Waldo, application wherever a weightless recognizer fits and forgetting is required.

References

- Aleksander, I., Thomas, W. V., and Bowden, P. A. (1984). WISARD: A radical step forward in image recognition. *Sensor Review*, 4(3):120–124.
- Barthelmes, T. and Vidal, F. P. (2021). Where’s wally? a machine learning approach. In *Computer Graphics and Visual Computing (CGVC)*, pages 28–31. The Eurographics Association.
- Bledsoe, W. W. and Browning, I. (1959). Pattern recognition and reading by machine. In *Papers Presented at the Eastern Joint IRE-AIEE-ACM Computer Conference*, pages 225–232. ACM.
- Bourtole, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., Travers, A., Zhang, B., Lie, D., and Papernot, N. (2021). Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 141–159. IEEE.
- Brophy, J. and Lowd, D. (2021). Machine unlearning for random forests. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 1092–1104. PMLR.

- Cao, Y. and Yang, J. (2015). Towards making systems forget with machine unlearning. In 2015 IEEE Symposium on Security and Privacy (SP), pages 463–480. IEEE.
- Carlini, N., Chien, S., Nasr, M., Song, S., Terzis, A., and Tramèr, F. (2022). Membership inference attacks from first principles. In 2022 IEEE Symposium on Security and Privacy (SP), pages 1897–1914. IEEE.
- Chen, M., Zhang, Z., Wang, T., Backes, M., Humbert, M., and Zhang, Y. (2021). When machine unlearning jeopardizes privacy. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS), pages 896–911. ACM.
- Constantinou, V. (2017). Hey, Waldo: Labeled images of the Where’s Waldo puzzle. <https://github.com/vc1492a/Hey-Waldo>. GitHub repository.
- Hayes, J., Shumailov, I., Triantafillou, E., Khalifa, A., and Papernot, N. (2024). Inexact unlearning needs more careful evaluations to avoid a false sense of privacy. arXiv preprint arXiv:2403.01218.
- Laguna, S., da Silva Gonçalves, J., Vandenhirtz, M., Ryser, A., Cannistraci, I., and Vogt, J. E. (2026). Rethinking machine unlearning: Models designed to forget via key deletion. In ICLR 2026 Workshop on Representational Structure and Intervention (RSI). arXiv:2603.15033.
- Lima Filho, A. S., Guarisa, G. P., Lusquino Filho, L. A. D., Oliveira, L. F. R., França, F. M. G., and Lima, P. M. V. (2020). wisardpkg: A library for WiSARD-based models. arXiv preprint arXiv:2005.00887.
- Lusquino Filho, L. A. D., Oliveira, L. F. R., Lima Filho, A. S., Guarisa, G. P., Felix, L. M., Lima, P. M. V., and França, F. M. G. (2020). Extending the weightless WiSARD classifier for regression. *Neurocomputing*, 416:280–291.
- Nguyen, T. T., Huynh, T. T., Nguyen, P. L., Liew, A. W.-C., Yin, H., and Nguyen, Q. V. H. (2022). A survey of machine unlearning. arXiv preprint arXiv:2209.02299.
- Shokri, R., Stronati, M., Song, C., and Shmatikov, V. (2017). Membership inference attacks against machine learning models. In 2017 IEEE Symposium on Security and Privacy (SP), pages 3–18. IEEE.
- Xu, H., Zhu, T., Zhang, L., Zhou, W., and Yu, P. S. (2023). Machine unlearning: A survey. *ACM Computing Surveys*, 56(1):1–36.